

New Kratos Developments

Carlos A .Roig CIMNE

 @roigcarlo



Overview

- FlowGraph
- The Registry
- The Multistage
- New C++ Testing



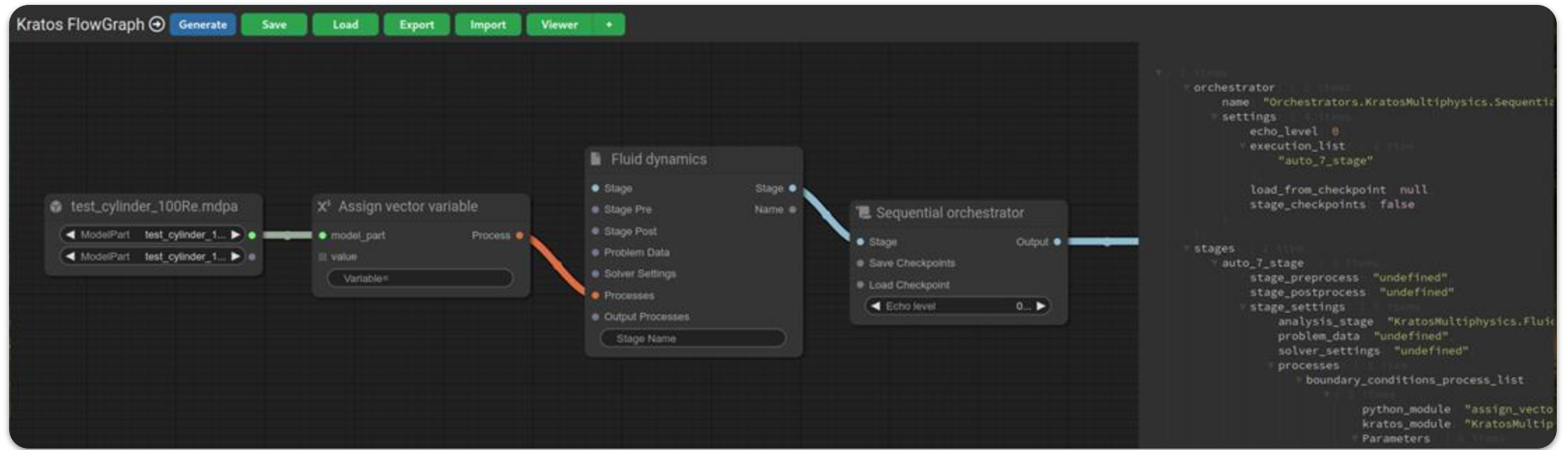
FlowGraph

What is FlowGraph



FlowGraph

- FlowGraph is a Node-Based programming tool to allow users to program complex Kratos simulations with ease written in JS.



FlowGraph

- Why we need this? Current Kratos Problem Type has limitations:
 - Difficult to make modifications (TCL)
 - Works only with GiD
 - Some advanced features do not match the current interface (Multistage)
- We seek a more modular system, which is easy to expand and integrable with other systems
- Compatible with current pre-post



FlowGraph

- Built to be easy to code:

```
class ImportMDPAModeler extends Modeler {
  constructor() {
    super()

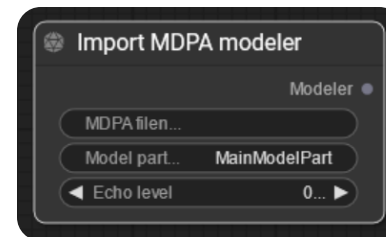
    // Model
    this.MODEL_INPUT = 0;
    this.MODEL_OUTPUT = 0;

    // Identifier Glyph
    this.glyph = {shape: '\uf6cf', font: '900 14px "Font Awesome 5 Free"', width: 16, height: 9};

    // List of inputs and outputs
    this.input_filename = this.addWidget("string", "MDPA filename", "", function(v){});
    this.model_part_name = this.addWidget("string", "Model part name", "MainModelPart", function(v){});
    this.echo_level = this.addWidget("combo", "Echo level", 0, function(v) {}, {
      values: [0, 1, 2, 3]
    });

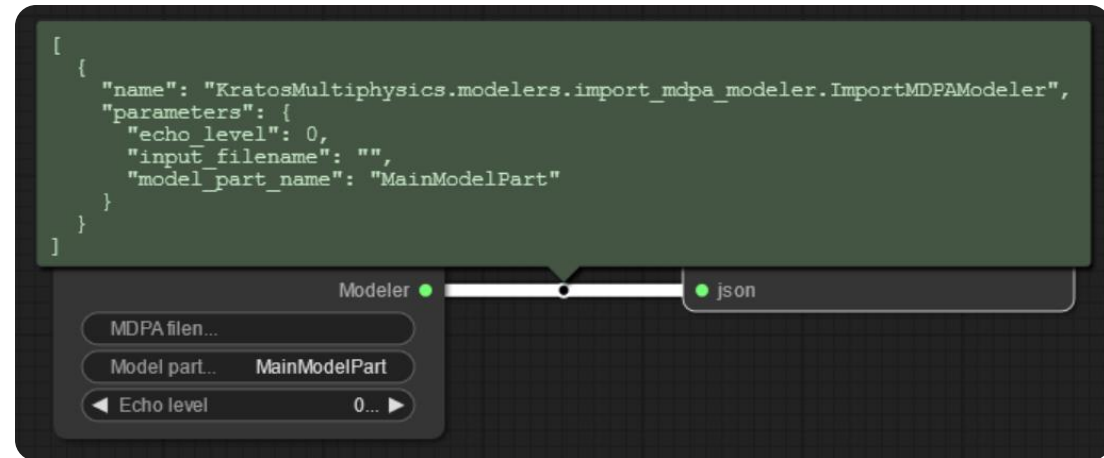
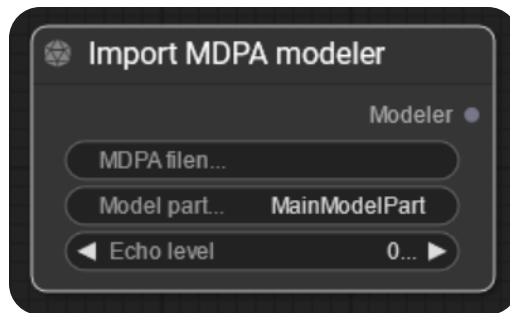
    this.size = this.computeSize();
    this.serialize_widgets = true;
  }

  onExecute() {
    let output = {
      "name" : "KratosMultiphysics.modelers.import_mdpa_modeler.ImportMDPAModeler",
      "parameters": {
        "echo_level" : parseInt(this.echo_level.value),
        "input_filename" : this.input_filename.value,
        "model_part_name" : this.model_part_name.value,
      }
    };
    this: this
    this.setOutputData(0, [output]);
  }
}
```



FlowGraph

- Built to be simple to use:



FlowGraph

- We allow us to generate ProjectParameters.json with ease.
- Visual programming means no previous coding knowledge needed.
- Moves the complexity from the user to the developer:
 - Explore models and see what connects with other nodes.



New Design

FlowGraph



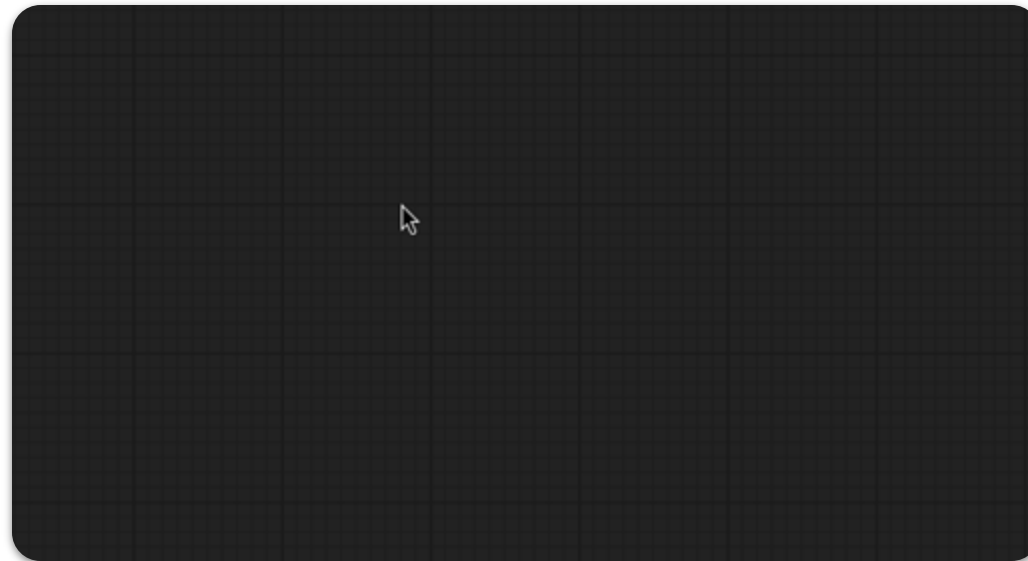
FlowGraph

- Previous versions of Flowgraph were shipped as static web application, each node needed to be integrated individually and only could be executed locally.
- We migrated the implantation to ES6 modules:
 - Default web-app still usable, but not possible to use as a module in a large ecosystem
 - Registration of nodes is done dynamically at runtime
 - Can run locally and in cloud based environments
 - Still based on <https://github.com/jagenjo/litegraph.js/>



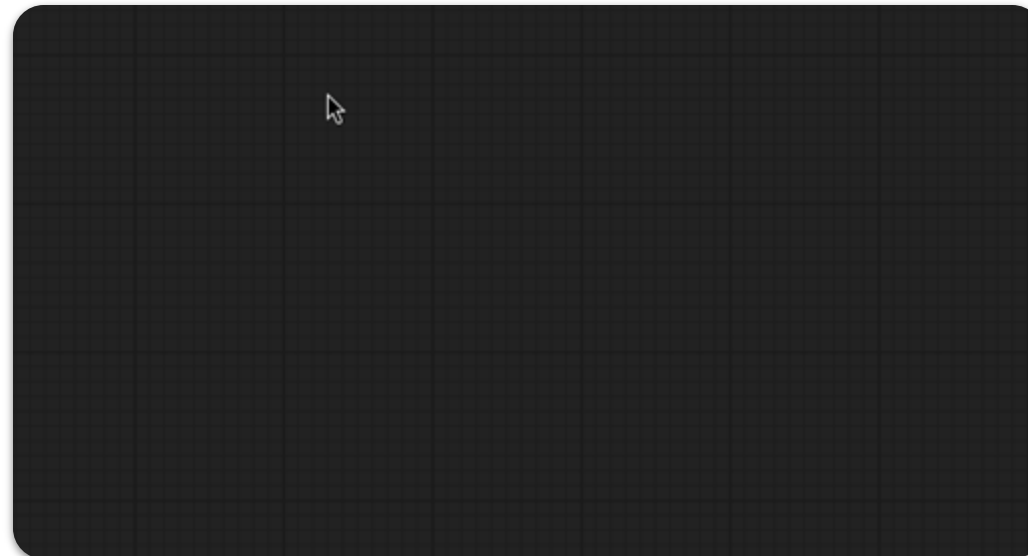
FlowGraph

- Since last shown, we have added a bunch of new nodes:
 - Solvers & Strategies
 - Processes & Materials
- Added support for Node Suggestion



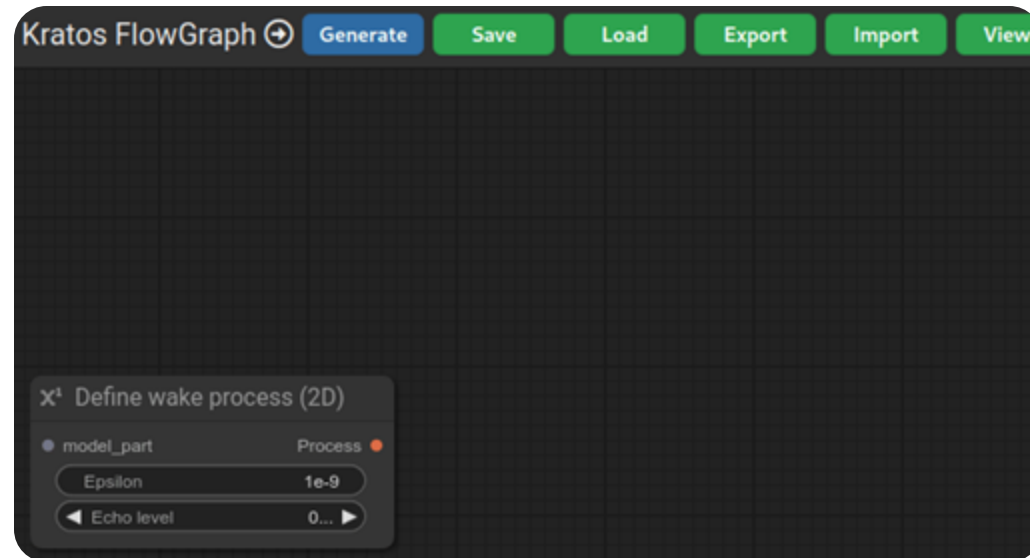
FlowGraph

- Added support to have node description, components hints, and links to Kratos doc directly inside the node
- We have added support to fully parse GiD generated models.



FlowGraph

- Now is possible to inspect pipelines to check intermediate json values (for example: debugging)
- Flowgraph can now be used to launch Kratos directly (WIP)



FlowGraph

- Currently mature support for FluidDynamics Application, Modelrs, Multistage and Orchestrator
- Working on adding nodes for more applications, with focus on Structural Mechanics
- Working on adding support to ProjectParameters.json -> FlowGraph (opposite operation)



Demo Time

FlowGraph



Registry

- Kratos has lots of different components: Elements, Processes, Solvers, Variables,
- Typically, it has been difficult to keep track of everything
 - Variables are registered in the core using KratosComponents
 - Processes and Utilities are floating around
 - We need one container for every entity registered, and those are not centralized in the same file
 - Python layer has no track of anything
- Registry comes as a solution to unify this behaviour
- Having a registry enables for more complex data flows -> Multistage



Registry

- The Goal: To have a centralized database where Kratos entities can be located
- It comes in two levels: C++ & Python
 - Python is able to access components from C++, but not the other way around
- Content of the registry is JSON serializable
 - Useful for serialization & debugging



Registry

- Everything is awesome! No.
- Problem 1: The entities we pretend to store are heterogeneous:
 - Not a problem in Python
 - Nightmare in C++
- Problem 2:
 - Register things manually is dull.
 - Register things automatically is hard.
- What do we do?



Registry

- Store Items in a generic Container called RegistryItem.
 - Underlying implementation uses `std::any` to be type agnostic
- Different types of entries:
 - Keys: Nothing to store
 - Simple values: Stored directly
 - Complex classes: Stored through class prototyping



Registry

```
KRATOS_TEST_CASE_IN_SUITE(RegistryItemGetValue, KratosCoreFastSuite)
{
    auto pProcess = Registry::GetValue<Process>("Processes.KratosMultiphysics.Process.Prototype");

    KRATOS_EXPECT_TRUE((std::is_base_of<Process, decltype(pProcess)>::value))
}
```

```
KRATOS_TEST_CASE_IN_SUITE(RegistryJsonValue, KratosCoreFastSuite)
{
    double value = 3.14;
    RegistryItem value_registry_item("value_item", value);

    std::string value_item_json = R"({
        "value_item": "3.14"
    })";

    KRATOS_EXPECT_EQ(value_registry_item.ToJson(), value_item_json);
}
```



Registry

- In Python keys are organized in the `python_registry.py` and managed automatically:

```
applications > FluidDynamicsApplication > python_registry_lists.py > ...
```

```
1 python_modelers_to_be_registered = []
2
3 python_operations_to_be_registered = []
4
5 python_processes_to_be_registered = [
6     "apply_boussinesq_force_process.ApplyBoussinesqForceProcess",
7     "apply_inlet_process.ApplyInletProcess"
8 ]
9
10 python_stages_to_be_registered = [
11     "fluid_dynamics_analysis.FluidDynamicsAnalysis"
12 ]
13
14 python_orchestrators_to_be_registered = []
```

```
registry_entry_point = f"{ItemKeyword}.{PythonModuleName}.{class_name}"
registry_entry_data = {
    "ClassName" : f"{class_name}",
    "ModuleName" : f"{module_name}"
}
KratosMultiphysics.Registry.AddItem(registry_entry_point, registry_entry_data)
```

```
class_name = registry_entry.split(".")[-1]
module_name = KM.Registry[f"{registry_entry}.ModuleName"]
module = import_module(module_name)
```



Registry

- In C++ you can access the registry directly to add anything you want:

```
#ifdef KRATOS_REGISTER_GEOMETRY
#undef KRATOS_REGISTER_GEOMETRY
#endif
#define KRATOS_REGISTER_GEOMETRY(name, reference)
    KratosComponents<Geometry<Node>>::Add(name, reference);
    if(!Registry::HasItem("geometries."+Registry::GetCurrentSource()+"."+name) &&
        !Registry::HasItem("components."+std::string(name))){
        Registry::AddItem<RegistryItem>("geometries."+Registry::GetCurrentSource()+"."+name);
        Registry::AddItem<RegistryItem>("components."+std::string(name));
    }
    Serializer::Register(name, reference);
```



Registry

- Classes can be registered automatically when the shared object is loaded through a macro in the class definition *
(not advisable to look at the implementation that macro)

```
KRATOS_REGISTRY_ADD_PROTOTYPE("Modelers.KratosMultiphysics", Modeler, CreateEntitiesFromGeometriesModeler)  
KRATOS_REGISTRY_ADD_PROTOTYPE("Modelers.All", Modeler, CreateEntitiesFromGeometriesModeler)
```



The MultiStage

Run Larger



Multistage

- Multistage is the Kratos solution for running simulations that need for different stages
- Introduced some new classes to Kratos
 - Project
 - Orchestrator
 - Operations



Multistage - Project

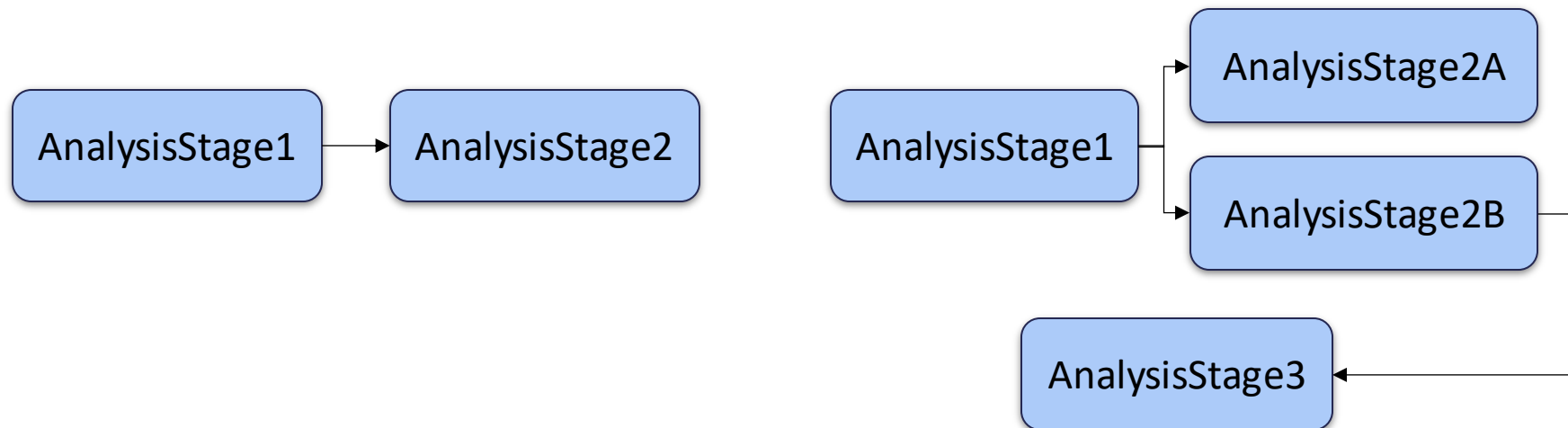
- Fully implemented in Python
- A higher level of abstraction, contains the Orchestrator, Model, Parameters of the simulation and generated Data.
- Responsible to call the orchestrator and handle the serialization of the simulation status: Current stage, generated data, etc...

```
> def GetModel(self) -> KratosMultiphysics.Model: ...  
> def GetSettings(self) -> KratosMultiphysics.Parameters: ...  
> def GetOutputData(self) -> dict: ...  
> def GetActiveStages(self) -> dict: ...  
> def AddActiveStage(self, stage_name: str, stage_instance: ...  
> def RemoveActiveStage(self, stage_name: str) -> None: ...  
> def Save(self, save_folder_path: pathlib.Path, checkpoint: ...  
> def Load(self, loading_path: pathlib.Path) -> None: ...
```



Multistage - Orchestrator

- Implemented in Python
- Handles the execution order and life cycle of the AnalysisStages
- From SequentialOrchestrator to user defined ones.



Multistage - Orchestrator

- Fully configurable from JSON

```
"orchestrator" : {  
  "name" : "Orchestrators.KratosMultiphysics.SequentialOrchestrator",  
  "settings" : {  
    "echo_level" : 0,  
    "execution_list" : ["fluid_stage", "thermal_stage"],  
    "load_from_checkpoint" : null,  
    "stage_checkpoints" : false  
  }  
},
```



Multistage - Operations

- In a nutshell: Processes for multistage
 - One entry point
 - Executed during
 - stage_preprocess
 - stage_postprocess

```
"operations" : [{  
    "name" : "custom_transfer_operation.CustomTransferOperation",  
    "parameters" : {  
        "input_model_part_name" : "FluidModelPart.Fluid",  
        "output_model_part_name" : "ThermalModelPart.Fluid"  
    }  
}]
```

```
import KratosMultiphysics  
  
class CustomTransferOperation(KratosMultiphysics.Operation):  
  
    def __init__(self, Model, Settings):  
        KratosMultiphysics.Operation.__init__(self)  
  
        self.model = Model  
        self.input_model_part_name = Settings["input_model_part_name"].GetString()  
        self.output_model_part_name = Settings["output_model_part_name"].GetString()  
  
    def Execute(self):  
        input_model_part = self.model.GetModelPart(self.input_model_part_name)  
        output_model_part = self.model.GetModelPart(self.output_model_part_name)  
  
        for node_in, node_out in zip(input_model_part.Nodes, output_model_part.Nodes):  
            velocity = node_in.GetSolutionStepValue(KratosMultiphysics.VELOCITY, 0)  
            node_out.SetSolutionStepValue(KratosMultiphysics.VELOCITY, 0, velocity)
```



New C++ Testing

GTest



New C++ Testing

- Old Kratos C++ testing was a bit outdated
 - Homebrew
 - Relied on python to be executed
 - Tests integrated with the code library
 - No isolation
- We considered several options, GTest was chosen
 - Widely used and supported
 - Advanced features (Gmock, filters, ...)
 - Transition was “easy”
- Joint effort between CIMNE & Deltares
- Triggered some changes in the core (and headaches)



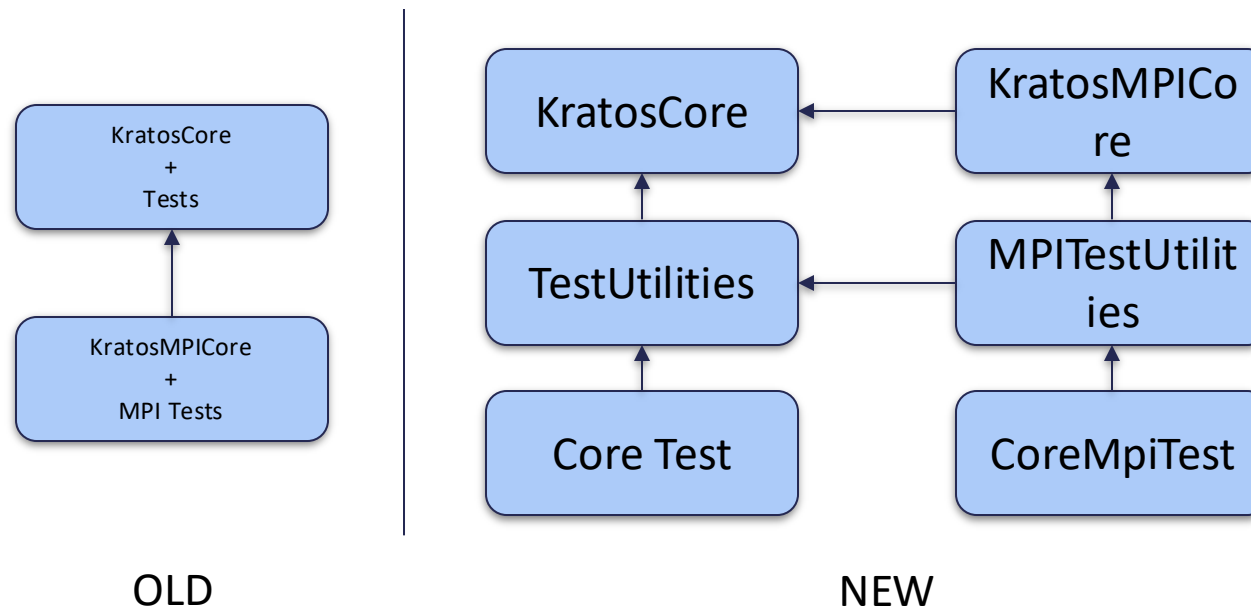
New C++ Testing

- Automatically downloaded when Kratos compiles
- One binary is generated per application and execution model:
 - KratosCoreTest
 - KRatosMPICoreTest
- Easy to setup with a custom macro



New C++ Testing

- Kratos building blocks have change a bit due to the tests:

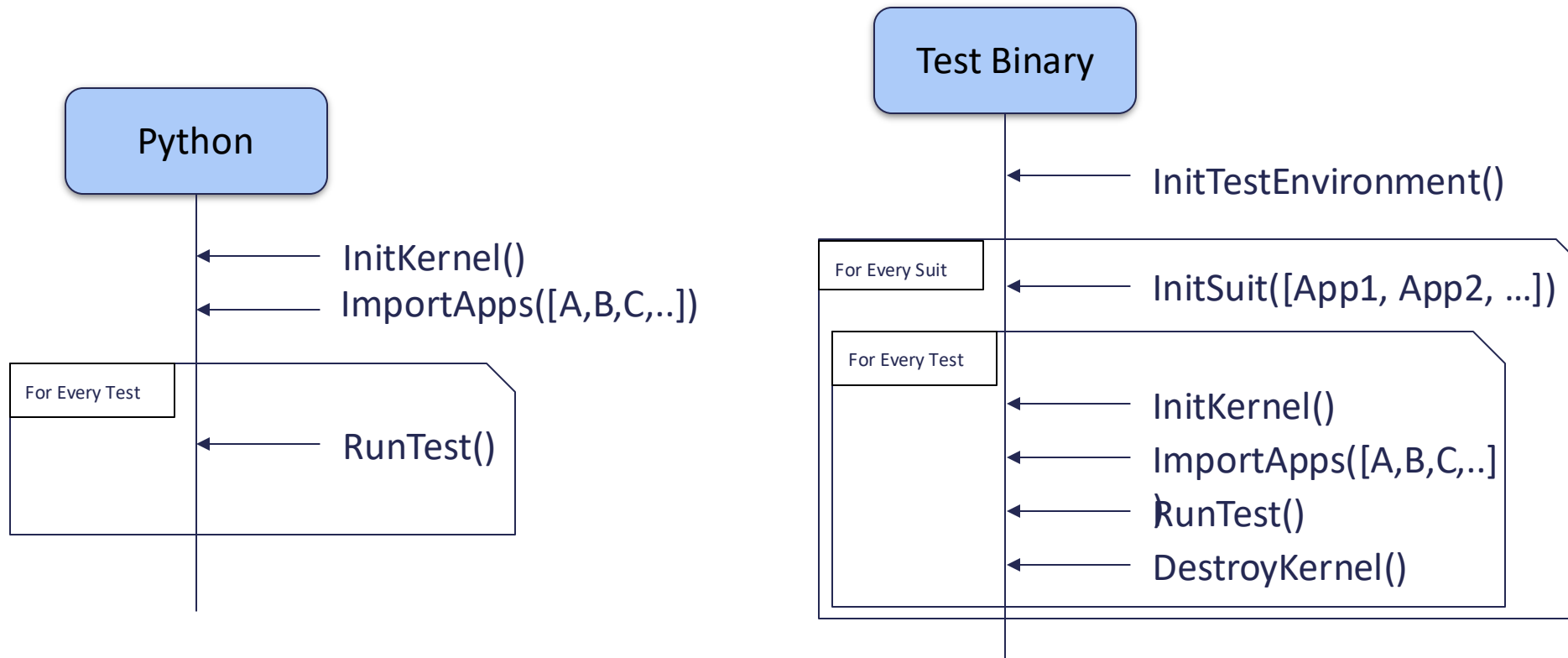


New C++ Testing

- Tests moved from being a functions to live in an Testing environment
- Environment controls properties of the test and life cycle of its components (kernel, applications, etc...)
- Multiple suits can be used to organize tests like python
- **→ Each test now uses its own kernel instance ←**
- One a suite is created, a list of applications needed is gathered and then each tests sets up a kernel and registers the needed applications



New C++ Testing



New C++ Testing

- At application level this changes means you now have a macro in CMake and a new class in the code:

```
## define fluid dynamics test sources
if(${KRATOS_BUILD_TESTING} MATCHES ON)
    file(GLOB_RECURSE KRATOS_FLUID_DYNAMICS_TESTING_SOURCES ${CMAKE_CURRENT_SOURCE_DIR}/tests/cpp_tests/*.cpp)
    kratos_add_gtests(TARGET KratosFluidDynamicsCore SOURCES "${KRATOS_FLUID_DYNAMICS_TESTING_SOURCES}")
endif(${KRATOS_BUILD_TESTING} MATCHES ON)
```

```
// Project includes
#include "fluid_dynamics_fast_suite.h"

namespace Kratos::Testing
{
    FluidDynamicsApplicationFastSuite::FluidDynamicsApplicationFastSuite()
        : KratosCoreFastSuite()
    {
        mpFluidDynamicsApp = std::make_shared<KratosFluidDynamicsApplication>();
        this->ImportApplicationIntoKernel(mpFluidDynamicsApp);
    }

} // namespace Kratos::Testing
```



Summary



Summary

- FlowGraph GUI is much more mature now.
- Registry is finished and started to being used
 - Fully removing KratosComponents will still take time
- First prototype of the multistage is finished and working
- GTest finally implemented and all tests have been migrated





 <https://github.com/KratosMultiphysics>

 www.youtube.com/@kratosmultiphysics3578

